Міністерство освіти та науки України

Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки

Кафедра інформатики та програмної інженерії

ЗВІТ

з дисципліни: «Програмування веб-застосувань»
Практична робота №13
Односторінкові застосунки (SPA) та клієнтська маршрутизація

Виконав:
Студент II курсу
групи ІП-56
Гончар І.В.

Перевірив:
Ковтунець О.В.

Київ 2026

1. Мета роботи:

Перетворити застосунок на просту SPA (Single Page Application): реалізувати клієнтську маршрутизацію без фреймворку — на вибір hash-based роутинг (location.hash) чи History API (pushState/popstate) — і рендерити різні «сторінки» застосунку в один контейнер без перезавантаження браузера. Щонайменше 3 маршрути (у тому числі один із параметром на кшталт /items/:id) мають повторно використовувати вже написані функції рендеру (практикум 7) і дані з localStorage/IndexedDB (практикум 11), а не дублювати розмітку чи логіку.

2. Хід виконання роботи:

Кроки 1–4. Обрати підхід, контейнер та таблицю маршрутів:

Для реалізації SPA було обрано підхід Hash-based роутингу (використання location.hash), що працює без додаткового налаштування сервера. У файлі index.html виділено єдиний контейнер , у який динамічно рендериться вміст активного виду. У файлі script.js описано таблицю маршрутів routes із трьома маршрутами (головна '/', деталі гри '/games/:id', та колекція '/owned') та реалізовано функцію matchRoute(path) для порівняння поточного шляху та витягування параметрів id.

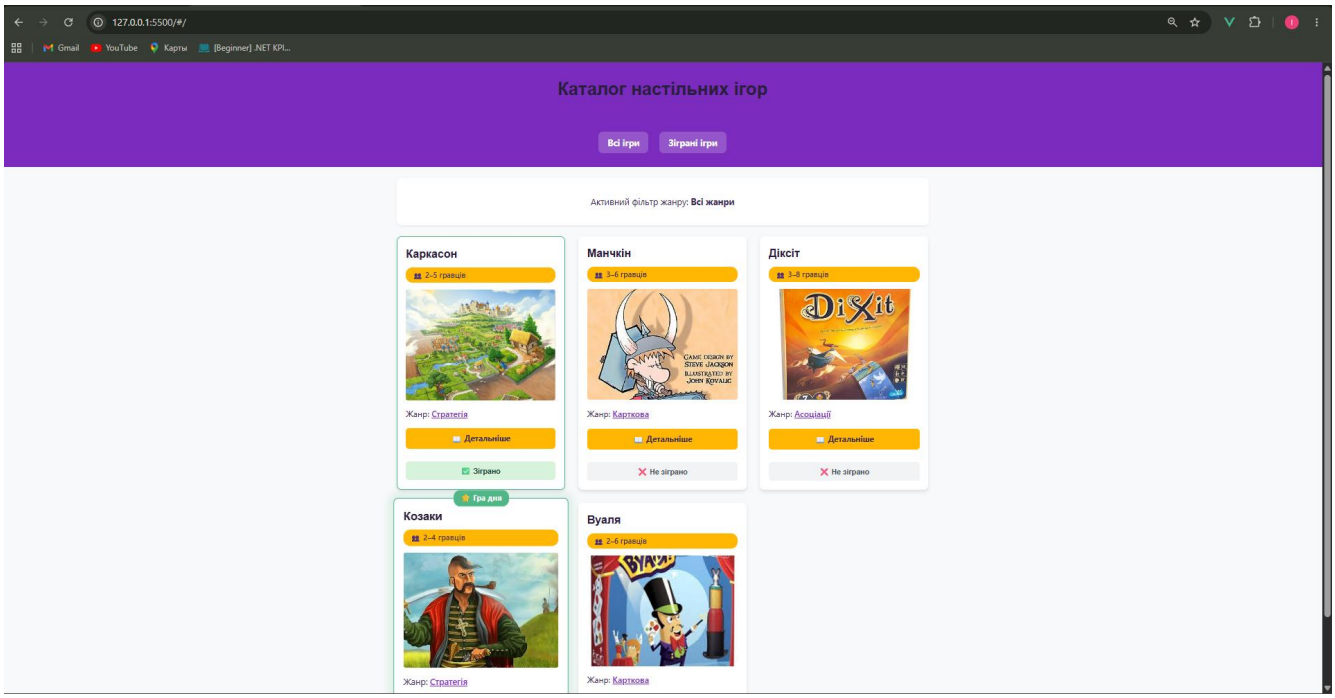


Рис. 1 — Головна сторінка каталогу (маршрут #/)

Кроки 5–7. View-компоненти, клієнтська навігація та hashchange:

Створено відповідні view-компоненти у Vue, які очищують та оновлюють контейнер залежно від обраного маршруту, повторно використовуючи логіку відображення карток з Практикуму 7 та збережені дані з IndexedDB (Практикум 11). Усі внутрішні посилання переведено на використовувати хеш-формат (href="#/", href="#/owned"). Оброблено подію hashchange, завдяки чому кнопки «назад» і «вперед» у браузері коректно перемальовують вміст сторінки.

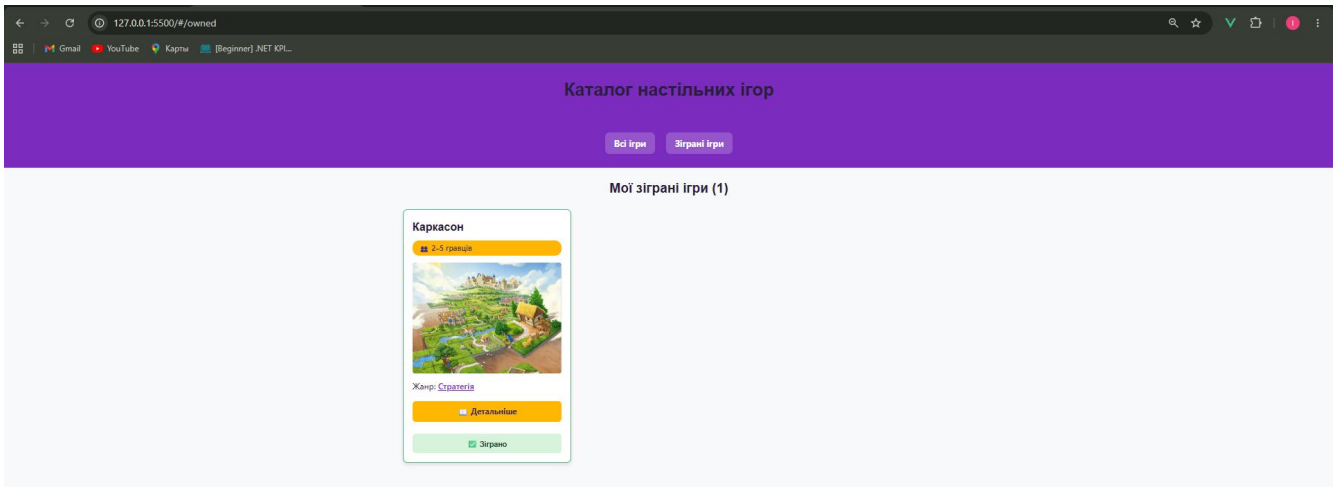


Рис. 2 — Розділ «Мої зіграні ігри» (маршрут #/owned)

Кроки 8–10. Сторінка деталей, 404 та перевірка навігації:

Реалізовано детальний перегляд картки гри за адресою #/games/:id. Компонент витягує id з параметрів шляху та знаходить потрібний запис у масиві даних з IndexedDB. Для неіснуючих маршрутів (наприклад, #/abc) реалізовано відображення сторінки 404 («Сторінку не знайдено») з посиланням на головну сторінку. Проведено тестування клієнтської навігації за допомогою кнопок браузера.

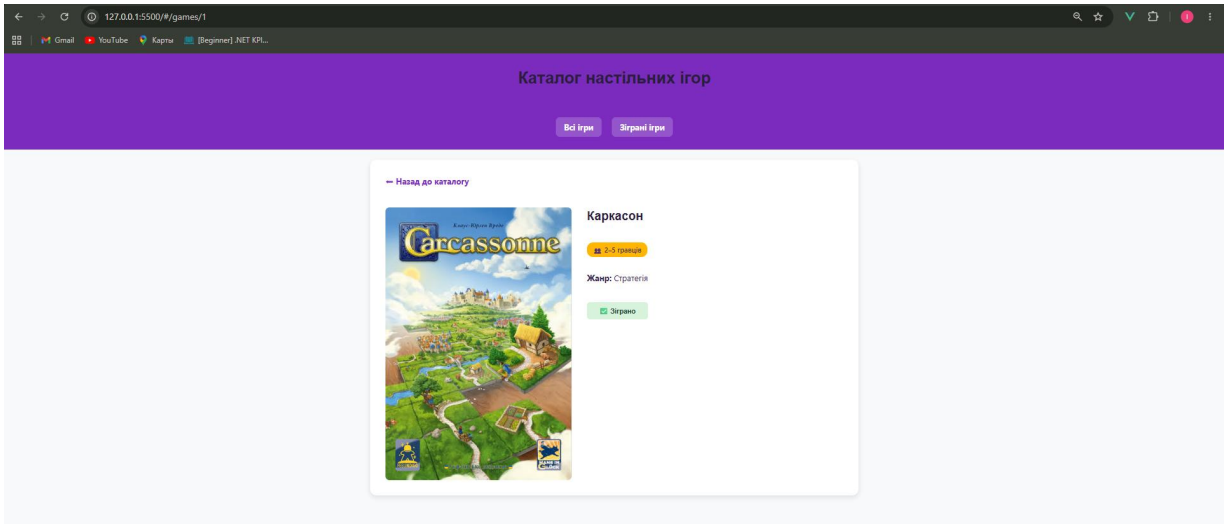


Рис. 3 — Сторінка деталей конкретної гри за ID (маршрут #/games/1)

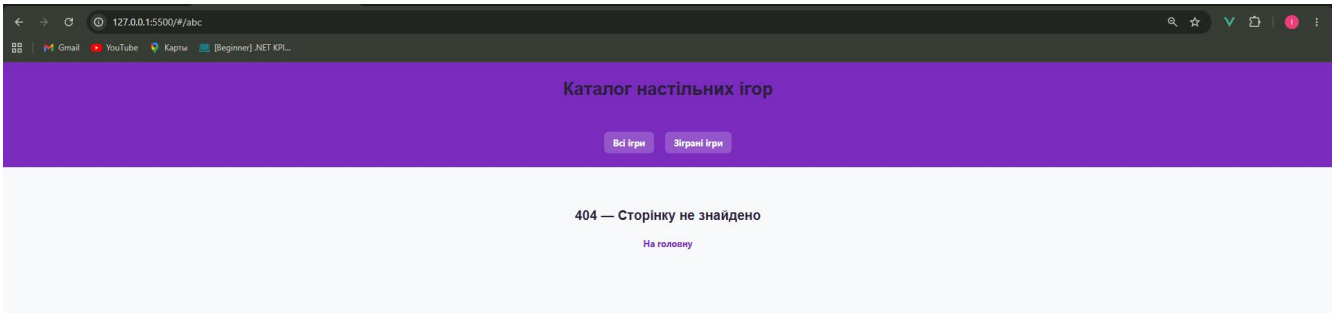


Рис. 4 — Відображення сторінки 404 при некоректному маршруті

Крок 11. Версіонування та збереження:

Проєкт збережено у локальному Git-репозиторії boardgame-spa, код прокоментовано, а зміни відправлено на GitHub.

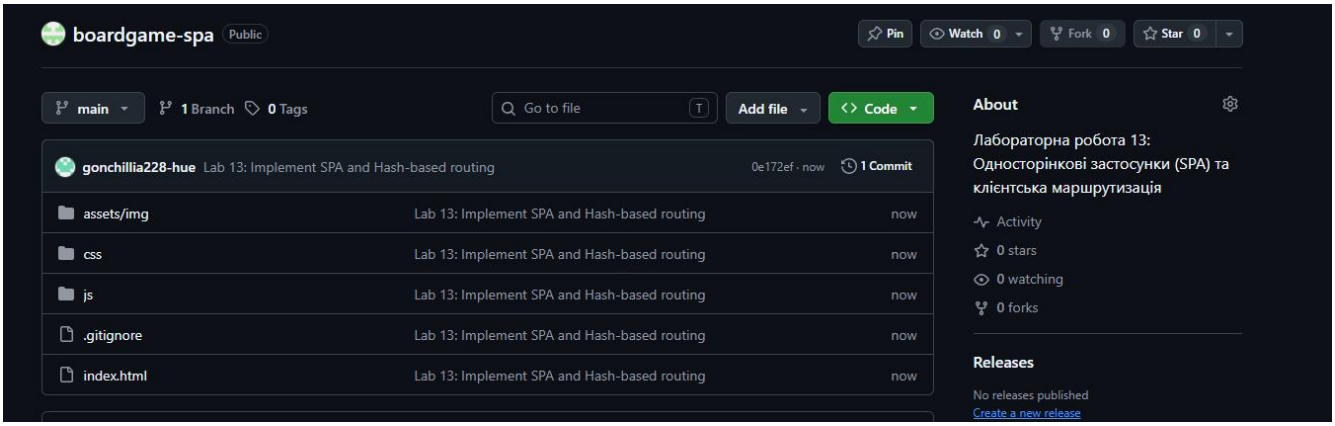


Рис. 5 — Фіксація змін та коміти у віддаленому репозиторії на GitHub

Посилання на репозиторій:

https://github.com/Bitshkiper218/boardgame-spa

Висновок:

Під час виконання практичної роботи №13 було успішно опановано принципи побудови односторінкових застосунків (SPA) та реалізовано клієнтську маршрутизацію на основі location.hash без використання сторонніх роутерів. Створено таблицю маршрутів та алгоритм matchRoute для обробки стандартних шляхів та динамічних параметрів (:id). Забезпечено перехід між каталогом ігор, детальним переглядом картки та фільтром зіграних ігор без перезавантаження документа, із повторним використанням розроблених у Практикумах 7 та 11 компонентів та бази даних IndexedDB. Реалізовано обробку 404 помилок та перевірено функціонування кнопок навігації браузера «назад/вперед». Фінальний код проєкту успішно збережено та завантажено до репозиторію на GitHub.